Natural Language Processing

Hugging Face Tutorial 2024/10/22

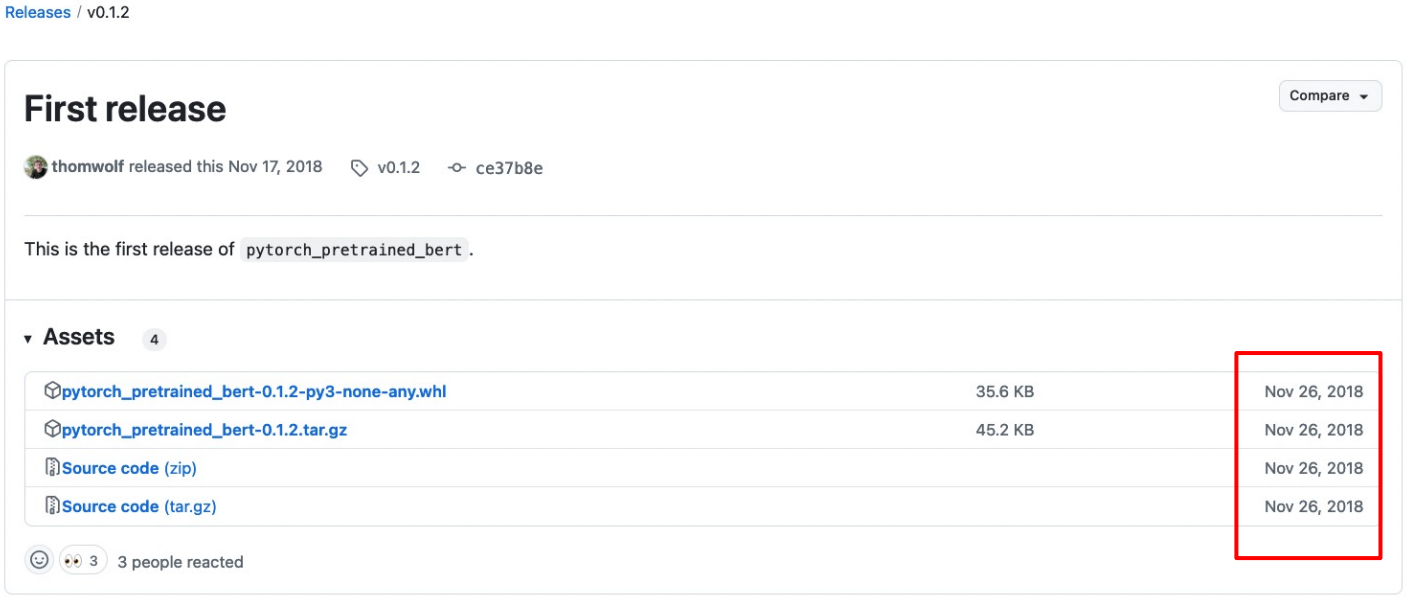
Outline

- Introduction to Hugging Face
- BERT
- Hugging Face Trainer

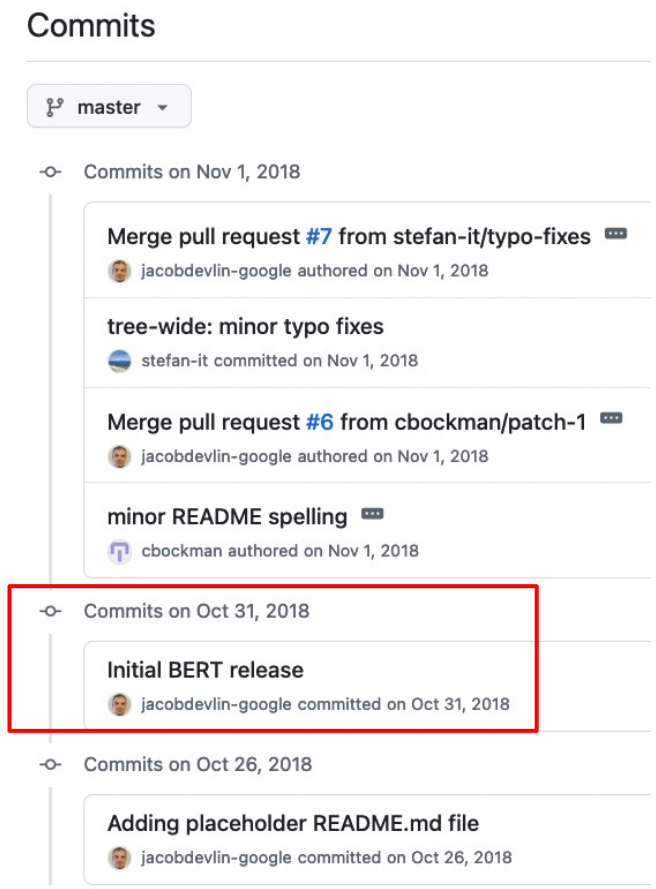
Introduction to Hugging Face

- An **open-source** library for state-of-the-art AI models (Natural Language Processing / Computer Vision / Audio ...) with various deep learning frameworks (PyTorch / TensorFlow / JAX ...)
- Links
  - Documentation: https://huggingface.co/docs
  - Source code: https://github.com/huggingface

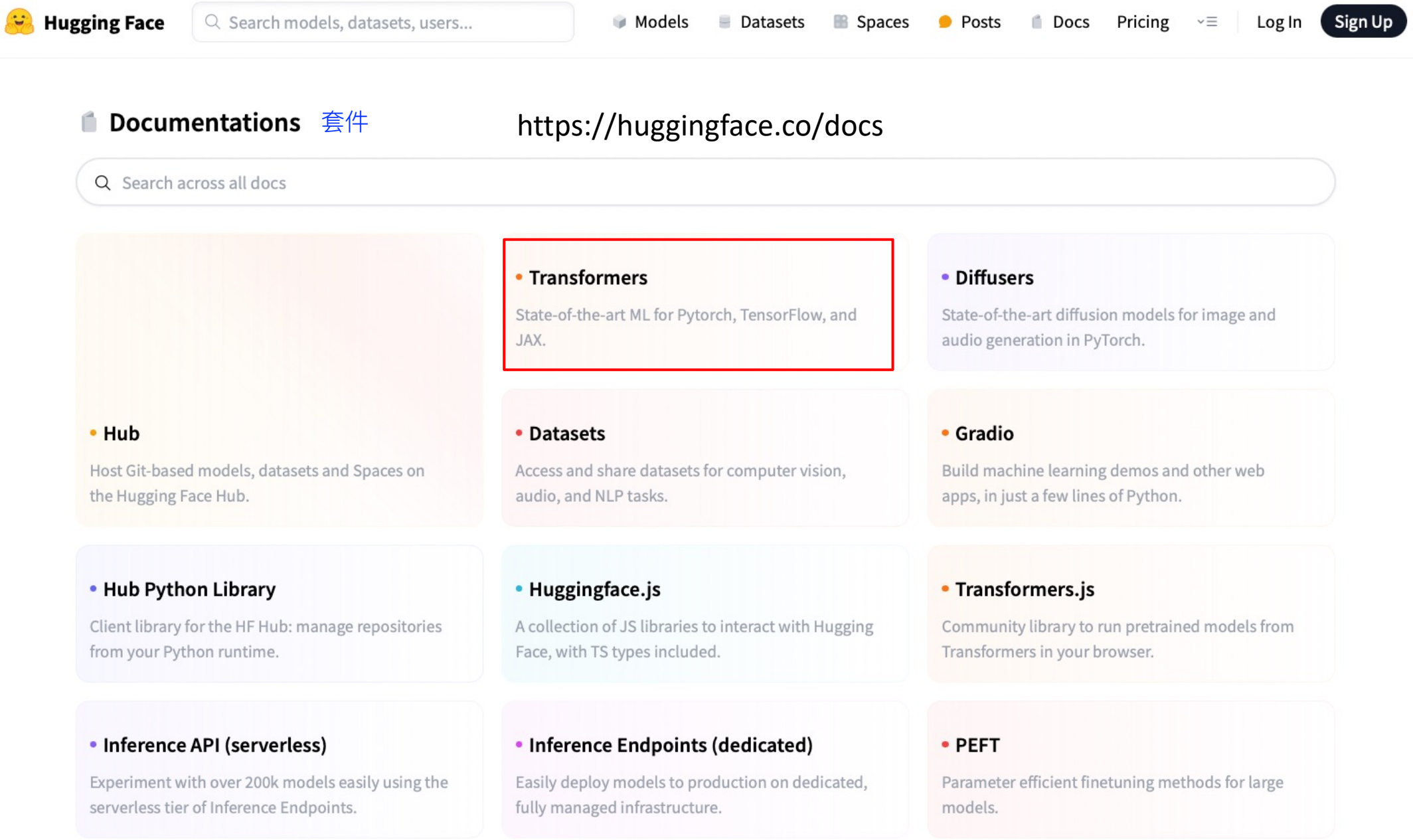
Hugging Face made the first PyTorch Implementation of BERT



https://github.com/huggingface/transformers/releases/tag/v0.1.2



https://github.com/google-research/bert/commits


Hugging Face
Search models, datasets, users...
Models
Datasets
Spaces
Posts
Docs
Pricing
Log In
Sign Up
Documentations
套件
https://huggingface.co/docs
Search across all docs
Transformers
State-of-the-art ML for Pytorch, TensorFlow, and JAX.
Diffusers
State-of-the-art diffusion models for image and audio generation in PyTorch.
Hub
Host Git-based models, datasets and Spaces on the Hugging Face Hub.
Datasets
Access and share datasets for computer vision, audio, and NLP tasks.
Gradio
Build machine learning demos and other web apps, in just a few lines of Python.
Hub Python Library
Client library for the HF Hub: manage repositories from your Python runtime.
Huggingface.js
A collection of JS libraries to interact with Hugging Face, with TS types included.
Transformers.js
Community library to run pretrained models from Transformers in your browser.
Inference API (serverless)
Experiment with over 200k models easily using the serverless tier of Inference Endpoints.
Inference Endpoints (dedicated)
Easily deploy models to production on dedicated, fully managed infrastructure.
PEFT
Parameter efficient finetuning methods for large models.

Installation

Basically, you need to install PyTorch first before you install transformers.

(Recommended)

```
pip install transformers
```

(If you want to try new things)

```
git clone
https://github.com/huggingface/transformers.git
cd transformers
pip install -e .
```

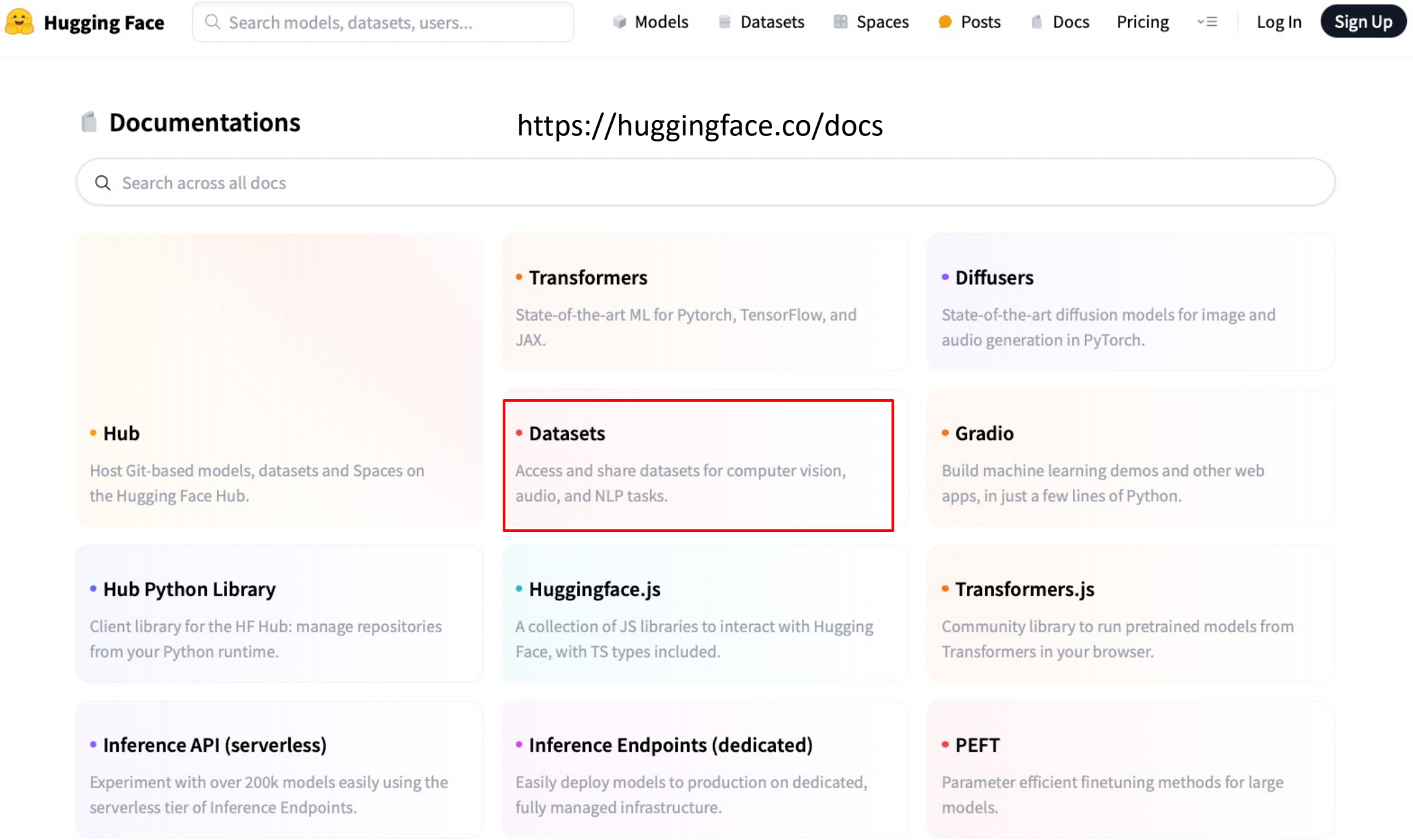
Packages required in this tutorial

We need the following packages for this tutorial. On Colab, you may not need to re-install these packages for they may have already been installed.

```
!pip install torch==2.4.0
!pip install transformers==4.37.0
!pip install datasets==3.0.1
!pip install accelerate==0.21.0
!pip install scikit-learn==1.5.2
!pip install wget
!pip install tarfile
```

模型評估

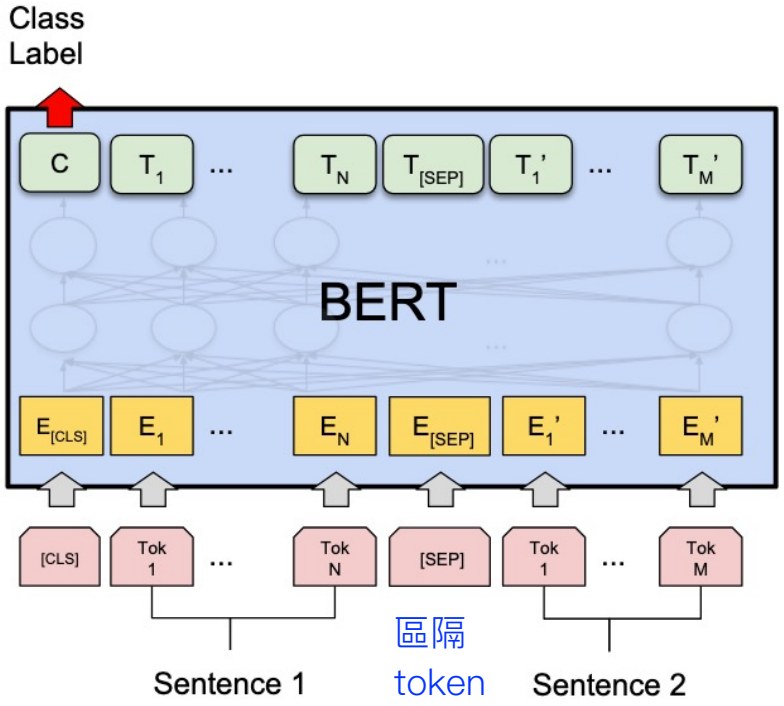
資料前處理套件


Hugging Face
Search models, datasets, users...
Models
Datasets
Spaces
Posts
Docs
Pricing
Log In
Sign Up
Documentations
https://huggingface.co/docs
Search across all docs
Transformers
State-of-the-art ML for Pytorch, TensorFlow, and JAX.
Diffusers
State-of-the-art diffusion models for image and audio generation in PyTorch.
Hub
Host Git-based models, datasets and Spaces on the Hugging Face Hub.
Datasets
Access and share datasets for computer vision, audio, and NLP tasks.
Gradio
Build machine learning demos and other web apps, in just a few lines of Python.
Hub Python Library
Client library for the HF Hub: manage repositories from your Python runtime.
Huggingface.js
A collection of JS libraries to interact with Hugging Face, with TS types included.
Transformers.js
Community library to run pretrained models from Transformers in your browser.
Inference API (serverless)
Experiment with over 200k models easily using the serverless tier of Inference Endpoints.
Inference Endpoints (dedicated)
Easily deploy models to production on dedicated, fully managed infrastructure.
PEFT
Parameter efficient finetuning methods for large models.

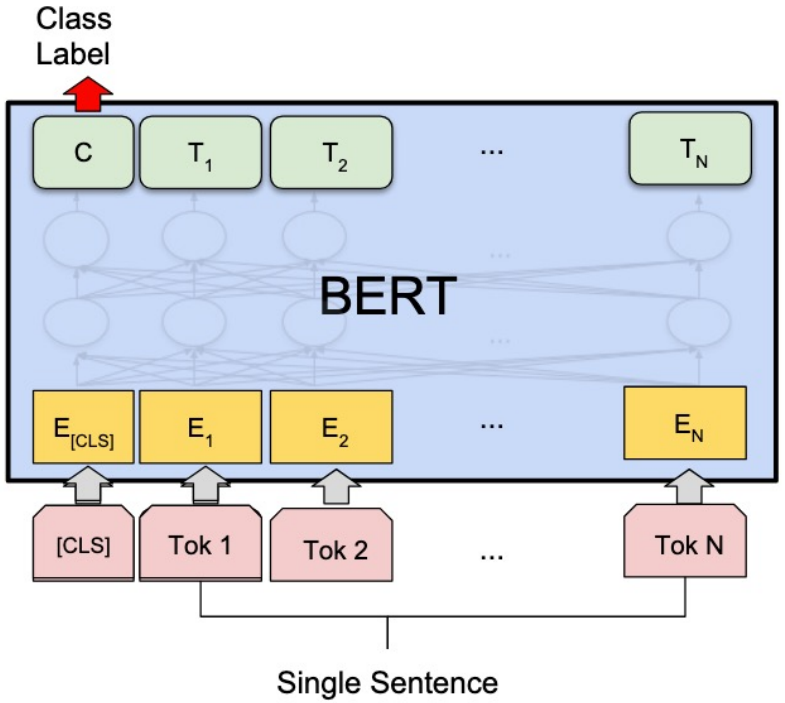
Check the versions of your packages

```
import torch
print(f"PyTorch version: {torch.__version__}")

import transformers
print(f"Hugging Face Transformers version: {transformers.__version__}")

import datasets
print(f"Hugging Face Datasets version: {datasets.__version__}")
```

Sentence Classification with BERT



(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG



(b) Single Sentence Classification Tasks: SST-2, CoLA 單一句子分類

Devlin et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." NAACL 2019.

BertForSequenceClassification

class transformers.**BertForSequenceClassification** <source>

```
( config )
```

Parameters

- **config** (BertConfig) — Model configuration class with all the parameters of the model. Initializing with a config file does not load the weights associated with the model, only the configuration. Check out the from_pretrained() method to load the model weights.

Bert Model transformer with a sequence classification/regression head on top (a linear layer on top of the pooled output) e.g. for GLUE tasks.

This model inherits from PreTrainedModel. Check the superclass documentation for the generic methods the library implements for all its model (such as downloading or saving, resizing the input embeddings, pruning heads etc.)

This model is also a PyTorch torch.nn.Module subclass. Use it as a regular PyTorch Module and refer to the PyTorch documentation for all matter related to general usage and behavior.

forward <source>

https://huggingface.co/docs/transformers/model_doc/bert#transformers.BertForSequenceClassification

Workflow for training a model using Hugging Face

Option1: Download the raw data
Option2: Get the dataset from Hugging Face Datasets (You may not need to perform data pre-processing.)

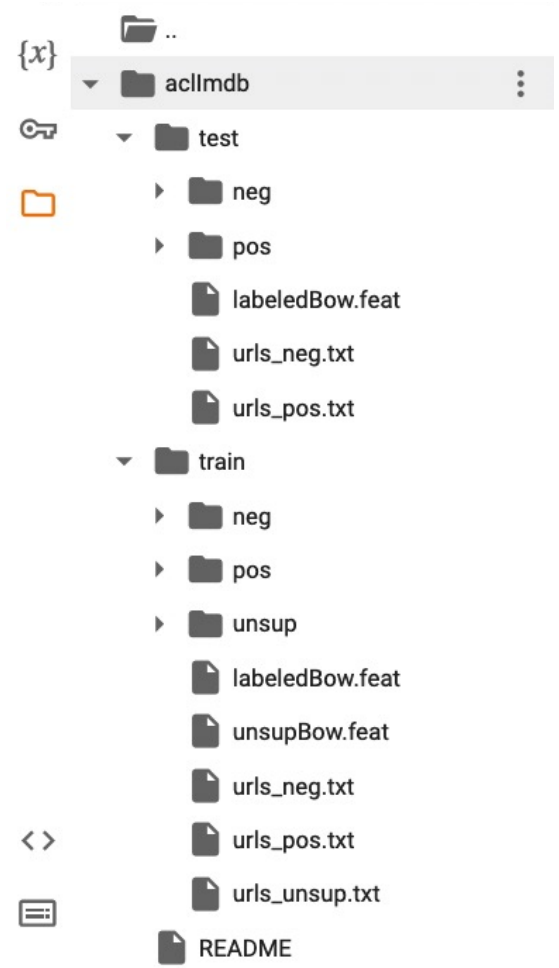
Option1: Use native PyTorch
Option2: Use Hugging Face Trainer

Option1: Use native PyTorch
Option2: Use Hugging Face Trainer

Dataset Preparation (IMDb reviews)

```
import wget
import tarfile

# Download the IMDb dataset
url = 'http://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz'
filename = wget.download(url, out='./')

# Unzip the `aclImdb_v1.tar.gz` file
tar = tarfile.open('aclImdb_v1.tar.gz', 'r:gz')
tar.extractall()
```

The folder structure of IMDb after unzipping



Sentiment Analysis

pos — Positive movie reviews

neg — Negative movie reviews

Create a function to pre-process the IMDb dataset

```python
# 5. Create a function to pre-process the IMDb dataset
def read_imdb_split(split_dir):
    split_dir = Path(split_dir)
    texts, labels = [], []
    for label_dir in ["pos", "neg"]:
        # Use glob() to get files with the extension ".txt"
        for text_file in (split_dir/label_dir).glob("*.txt"):
            # read_text() returns the decoded contents of the pointed-to file as a string
            tmp_text = text_file.read_text()

            # Append the read text to the list we defined in advance
            texts.append(tmp_text)

            # Build labels based on the folder name
            labels.append(0 if label_dir == "neg" else 1)

    return texts, labels
```

[Doc of Pathlib] https://docs.python.org/3/library/pathlib.html#pathlib.Path.read_text

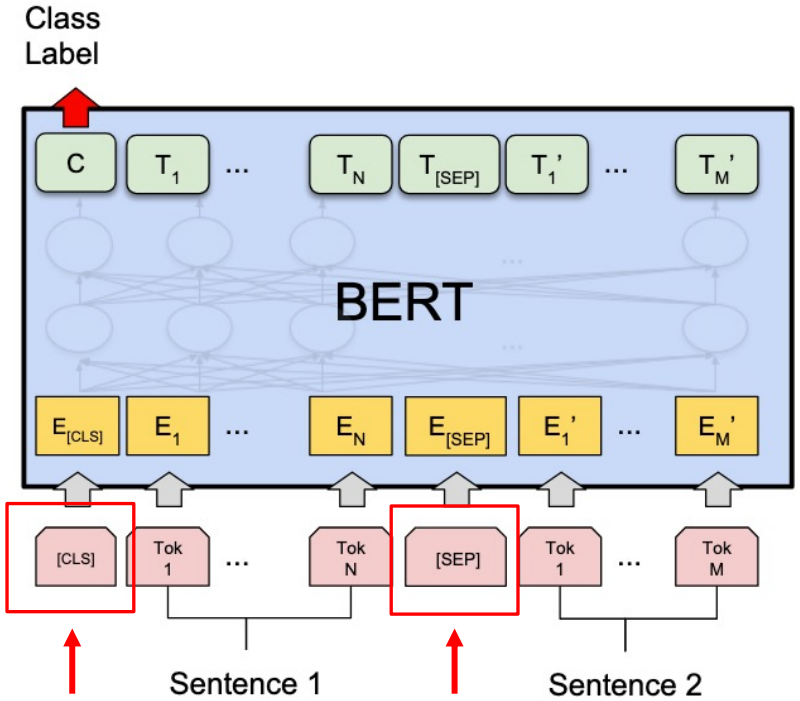
Pre-process the IMDb dataset

```
# 6. Pre-process the IMDb dataset (run the cell)

train_texts, train_labels = read_imdb_split('aclImdb/train')
test_texts, test_labels = read_imdb_split('aclImdb/test')
```

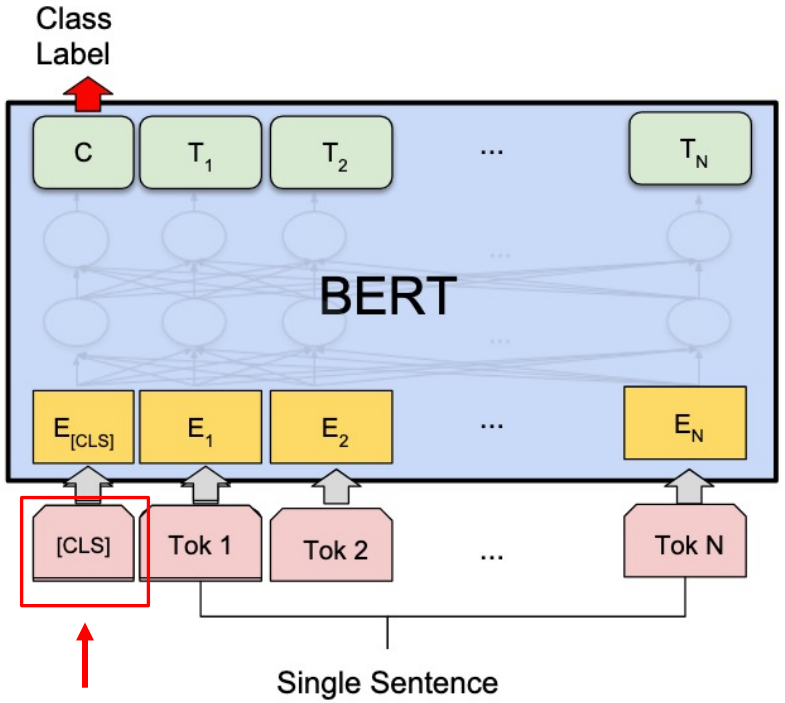
Dataset splitting

```python
# 7. Use train_test_split to split the training data into training and validation data
from sklearn.model_selection import train_test_split

# Set a random seed for reproducibility
random_seed = 42

# Set the ratio of the validation set to the training set
valid_ratio = 0.2

train_texts, val_texts, train_labels, val_labels = train_test_split(
    train_texts,
    train_labels,
    test_size=valid_ratio,
    random_state=random_seed
)
```

Pre-process the input sentences for BERT



(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



(b) Single Sentence Classification Tasks:
SST-2, CoLA

不管是單句或多句，實作時都會增加CLS/SEP special tokenizor

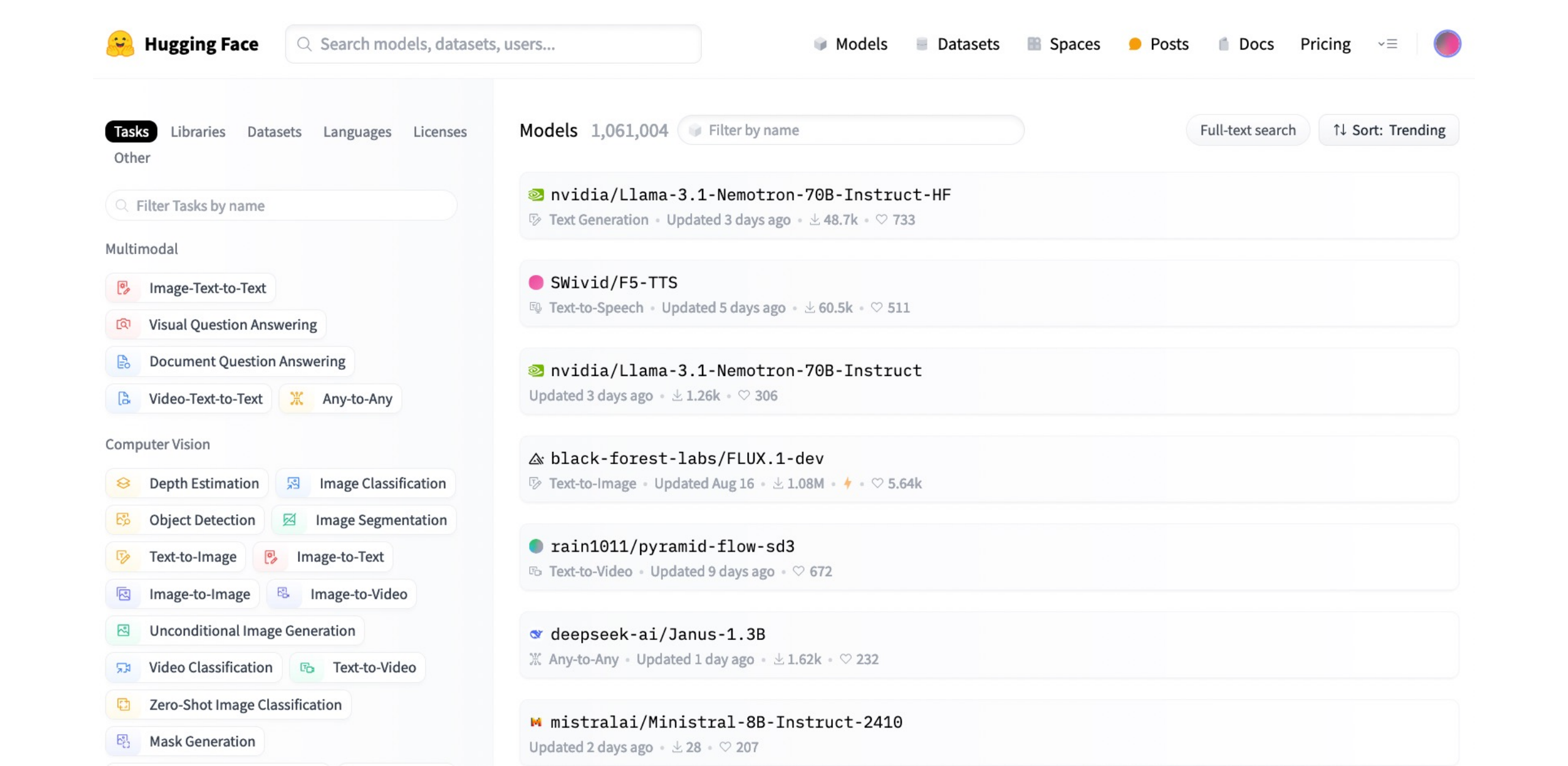
Devlin et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." NAACL 2019.

Hugging Face Tokenizer

```
# Load the Hugging Face tokenizer

model_name = "bert-base-uncased"
# Use .from_pretrained() for a pre-trained model
tokenizer = transformers.AutoTokenizer.from_pretrained(model_name)
```

uncased: it does not make a difference between english and English.

https://huggingface.co/google-bert/bert-base-uncased
https://huggingface.co/google-bert/bert-base-cased



https://huggingface.co/models

AutoClasses (models, tokenizers)

統一接口，只要用名字指定就可以直接使用

- BertTokenizer. GPT2Tokenizer
- In many cases, the architecture you want to use **can be guessed** from the name or the path of the pretrained model you are supplying to the from_pretrained method.
- AutoClasses are here to do this job for you so that you automatically retrieve the relevant model given the name/path to the pretrained weights/config/vocabulary:

- Ex: model = AutoModel.from_pretrained('bert-base-cased') will create an instance of BertModel).

https://huggingface.co/transformers/v3.0.2/model_doc/auto.html

Outputs of the Tokenizer (Case1)

truncation: 是否需要將long sequence切斷
padding: 每個batch是否需要統一長度–不夠長就補0

Do truncation and padding to the model_max_length

```
# 8. Load the Hugging Face tokenizer

train_encodings = tokenizer(train_texts, truncation=True, padding=True)
val_encodings = tokenizer(val_texts, truncation=True, padding=True)
test_encodings = tokenizer(test_texts, truncation=True, padding=True)
```

	Type_1	Type_2
truncation	True / False	longest_first /only_second / only_first / do_not_truncate=false
padding	True / False	longest / max_length / do_not_pad
max_length	None (model_max_length)	Your desired maximum length (integer).

only_first/second–只能用在 sentence pair classfication session

https://huggingface.co/docs/transformers/pad_truncation

Outputs of the Tokenizer (Case2)

Do truncation and padding to a length of 300 tokens

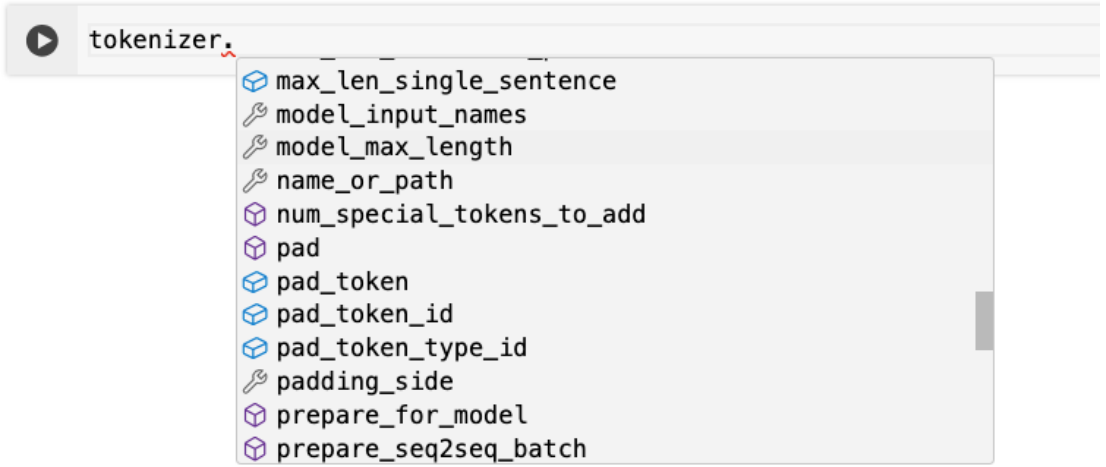
```
encodings = tokenizer(
        texts,
        truncation=True,
        padding=True,
        max_length=300
)
```

	Type_1	Type_2
truncation	True / False	longest_first /only_second / only_first / do_not_truncate
padding	True / False	longest / max_length / do_not_pad
max_length	None (model_max_length)	Your desired maximum length (integer).

https://huggingface.co/docs/transformers/pad_truncation

Methods inside `tokenizer`

- Type the dot and wait for a few seconds (on Colab)



- Or you can use `dir` to check all the methods of the object

```
tokenizer_methods = dir(tokenizer)

# Group them into rows for better readability
for i in range(0, len(tokenizer_methods), 5):
        print(", ".join(tokenizer_methods[i:i+5]))
```

Some useful methods of `tokenizer`

Example using `"bert-base-uncased"`

(From this slide, the output will be highlighted in blue.)

```
# Get the maximum length of the pre-trained model
print(tokenizer.model_max_length)
```

```
512
```

```
# Get the ID of the special tokens
print(tokenizer.vocab["[CLS]"])
print(tokenizer.vocab["[SEP]"])
print(tokenizer.vocab["[PAD]"])
```

```
101
102
0
```

Check the tokenized results

The output of `tokenizer` is a dictionary wrapper.

```
# See the keys of the tokenizer output
print(val_encodings.keys())
```

dict_keys(['input_ids', 'token_type_ids', 'attention_mask'])

```
# See which type the output encodings belong to
print(type(val_encodings))
```

transformers.tokenization_utils_base.BatchEncoding

https://huggingface.co/transformers/v3.5.1/_modules/transformers/tokenization_utils_base.html

Check the tokenized results (input_ids)

Check the first data of the validation set.

```
print(val_encodings.input_ids[0])
```

[CLS] [101, 1045, 2442, 6449, 1010, 1045, 2001, 2028, 1997, 1996, 15315, 23606, 6558, 2040, 28179, 13224, 2023, 2265, 2077, 4659, 2151, 2592, 2001,
4487, 11393, 26972, 2055, 2009, 1012, 1045, 4340, 2008, 2009, 2001, 2183, 2000, 2022, 1037, 10036, 6714, 1011, 2125, 8546, 2011, 8923, 1040,
1012, 5405, 26974, 1996, 2128, 13535, 2239, 1011, 23967, 1012, 1026, 7987, 1013, 1028, 1026, 7987, 1013, 1028, 1045, 2001, 3308, 999, 1026,
7987, 1013, 1028, 1026, 7987, 1013, 1028, 1996, 4405, 3727, 2019, 6581, 8605, 2588, 1996, 7193, 1012, 1996, 23661, 2003, 28851, 999, 1997, 2607,
1010, 28223, 18667, 2290, 8244, 2097, 2424, 3209, 26275, 1999, 1996, 5436, 1010, 2029, 2003, 4208, 2006, 1996, 2458, 1997, 1996, 22330, 7811,
2015, 2077, 1996, 2034, 2162, 1012, 1006, 5388, 2086, 2077, 1996, 2824, 1997, 1996, 18667, 2290, 4405, 1007, 1012, 1996, 4405, 2036, 4473, 2005,
24159, 1010, 4415, 10886, 2049, 5436, 1998, 4784, 1999, 1996, 2034, 2112, 1997, 1996, 2792, 1012, 1026, 7987, 1013, 1028, 1026, 7987, 1013,
1028, 2123, 1005, 1056, 2022, 13534, 1024, 1000, 6178, 14735, 1000, 2003, 2025, 18667, 2290, 1012, 2057, 2024, 3591, 2007, 2019, 10047, 16862,
3512, 1010, 18439, 3689, 20384, 2011, 26422, 1010, 15236, 1010, 1998, 6801, 4784, 1012, 1026, 7987, 1013, 1028, 1026, 7987, 1013, 1028, 1996,
9179, 9319, 18667, 2290, 1005, 1055, 4781, 1025, 2358, 27914, 2480, 1998, 17103, 2024, 3432, 2004, 24826, 15683, 1012, 17103, 1005, 13954,
1997, 3312, 4205, 2050, 1010, 4427, 2011, 19330, 15530, 1005, 13954, 1997, 2520, 1010, 3957, 1037, 6919, 12185, 1997, 2520, 1005, 1055, 14779,
2269, 1012, 2358, 27914, 2480, 1005, 1055, 13954, 1997, 2852, 1012, 3897, 9221, 27895, 2015, 1037, 2843, 1997, 3241, 1998, 3980, 1012, 1026,
7987, 1013, 1028, 1026, 7987, 1013, 1028, 2065, 1996, 3737, 1997, 1996, 4405, 2003, 2151, 12407, 1997, 2054, 1005, 1055, 2664, 2000, 2272, 1010,
16428, 2213, 1998, 1996, 5541, 2136, 2024, 2275, 2000, 3613, 18667, 2290, 1005, 1055, 8027, 1997, 2034, 1011, 3446, 2547, 4730, 2007, 2178,
3040, 7699, 2580, 2547, 17743, 1012, 102, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

[SEP] [PAD]

Check the tokenized results (token_type_ids)

Check the first data of the validation set.

```
print(val_encodings.token_type_ids[0])
```

```
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

All zeros show that this is a single-sentence task.
There will be additional ones for the token places of a second sentence.

Check the tokenized results (attention_mask)

Check the first data of the validation set.

```
print(val_encodings.attention_mask[0])
```

[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

Padding (telling the model not to focus on these regions)

協助模型切出需要觀察的重要區域–沒有補0的部分(原本不存在的區域)

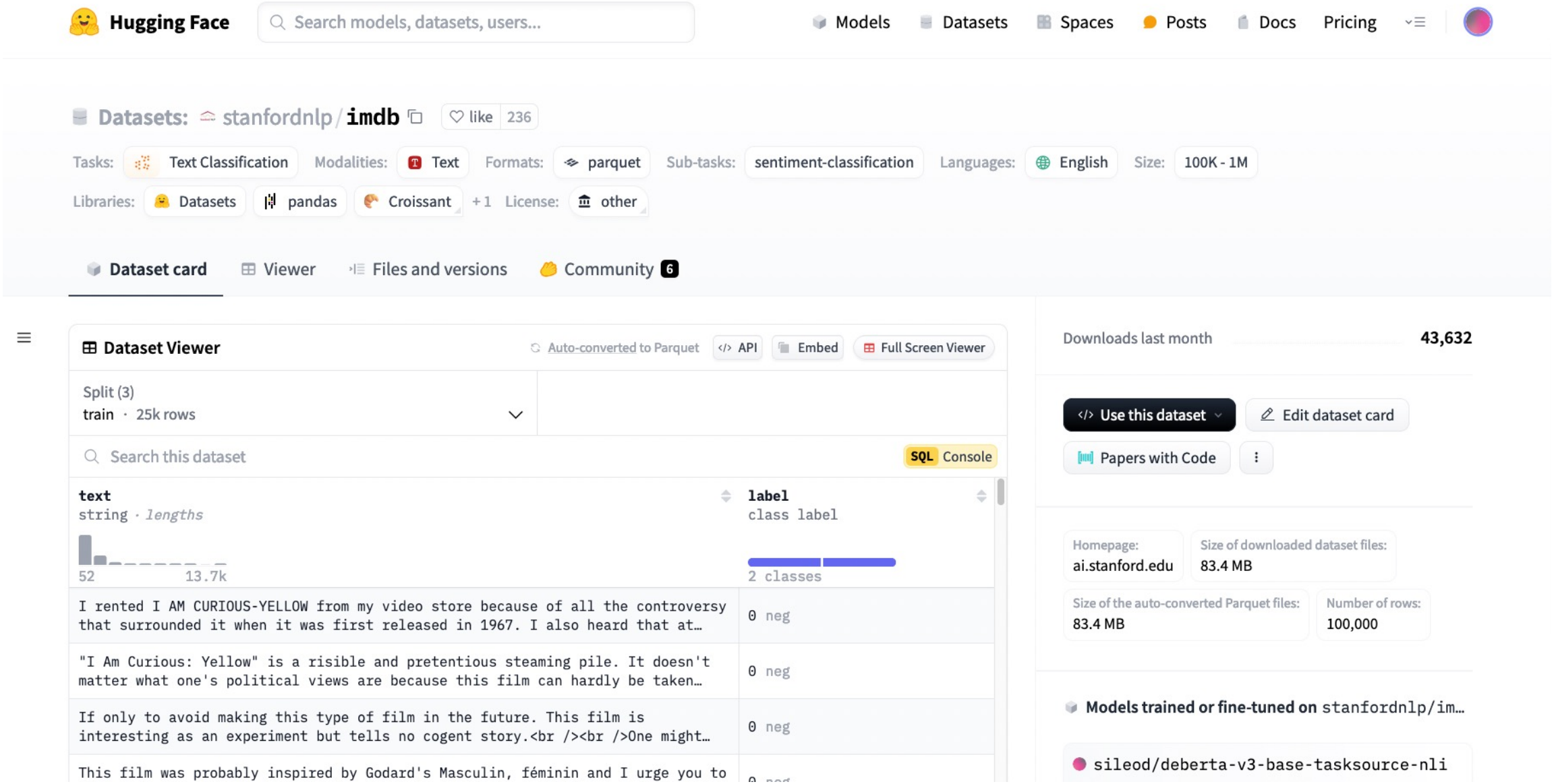
Building the **Dataset** class using PyTorch

```python
class IMDbDataset(torch.utils.data.Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __getitem__(self, idx):
        # Note that the tokenizer output is a dict wrapper
        # Convert data and labels into PyTorch tensors
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
        item['labels'] = torch.tensor(self.labels[idx])

        return item

    def __len__(self):
        # Number of a dataset
        return len(self.labels)

train_dataset = IMDbDataset(train_encodings, train_labels)
val_dataset = IMDbDataset(val_encodings, val_labels)
test_dataset = IMDbDataset(test_encodings, test_labels)
```

Or you can use Hugging Face Datasets

Using Hugging Face Dataset

```
# Load the IMDb training set
train = datasets.load_dataset("imdb", split="train")

# Split the validation set
random_seed = 42
splits = train.train_test_split(
test_size=0.2,
seed=random_seed
)
train, valid = splits['train'], splits['test']

# Load the IMDb test set
test = datasets.load_dataset("imdb", split="test")
```



https://huggingface.co/datasets/stanfordnlp/imdb

You still need to tokenize the dataset

Using Hugging Face Dataset

```python
def to_torch_data(hug_dataset):
    """Transform Hugging Face Datasets into PyTorch Dataset
    Args:
        - hug_dataset: data loaded from HF Datasets
    Return:
        - dataset: PyTorch Dataset
    """
    dataset = hug_dataset.map(
        lambda batch: tokenizer(
            batch["text"],
            truncation=True,
            padding=True
        ),
        batched=True
    )
    dataset.set_format(
        type='torch',
        columns=[
            'input_ids',
            'attention_mask',
            'label'
        ]
    )
    return dataset
```

Speed up processing (ref)

Set the format of the dataset to return PyTorch tensors instead of lists (ref)

pytorch所需要的模組建立

```python
train_dataset = to_torch_data(train)
val_dataset = to_torch_data(valid)
test_dataset = to_torch_data(test)
```

Load the BERT classification model

```
from transformers import AutoModelForSequenceClassification

model_name = "bert-base-uncased"
# Use .from_pretrained() for a pre-trained model
model = AutoModelForSequenceClassification.from_pretrained(
        model_name, num_labels=2
)
```

positive, negative

Different models for different tasks

Question Answering Tasks (SQuAD v1.1), ref

```
model = AutoModelForQuestionAnswering.from_pretrained(model_name)
```

Single Sentence Tagging Tasks (CoNLL-2003 NER; Named Entity Recognition), ref

```
model = AutoModelForTokenClassification.from_pretrained(model_name)
```

Semantic Textual Similarity Tasks (STS-B), ref

```
model = AutoModelForSequenceClassification.from_pretrained(
        model_name, num_labels=1
)
```

相似度任務–兩個句子的相似程度

What happens for down-stream tasks?

https://github.com/huggingface/transformers/blob/v4.45.2/src/transformers/models/bert/modeling_bert.py#L1651-L1665

```
1651  class BertForSequenceClassification(BertPreTrainedModel):
1652      def __init__(self, config):
1653          super().__init__(config)
1654          self.num_labels = config.num_labels
1655          self.config = config
1656
1657          self.bert = BertModel(config)
1658          classifier_dropout = (
1659              config.classifier_dropout if config.classifier_dropout is not None else config.hidden_dropout_prob
1660          )
1661          self.dropout = nn.Dropout(classifier_dropout)
1662          self.classifier = nn.Linear(config.hidden_size, config.num_labels)
1663
1664          # Initialize weights and apply final processing
1665          self.post_init()
```

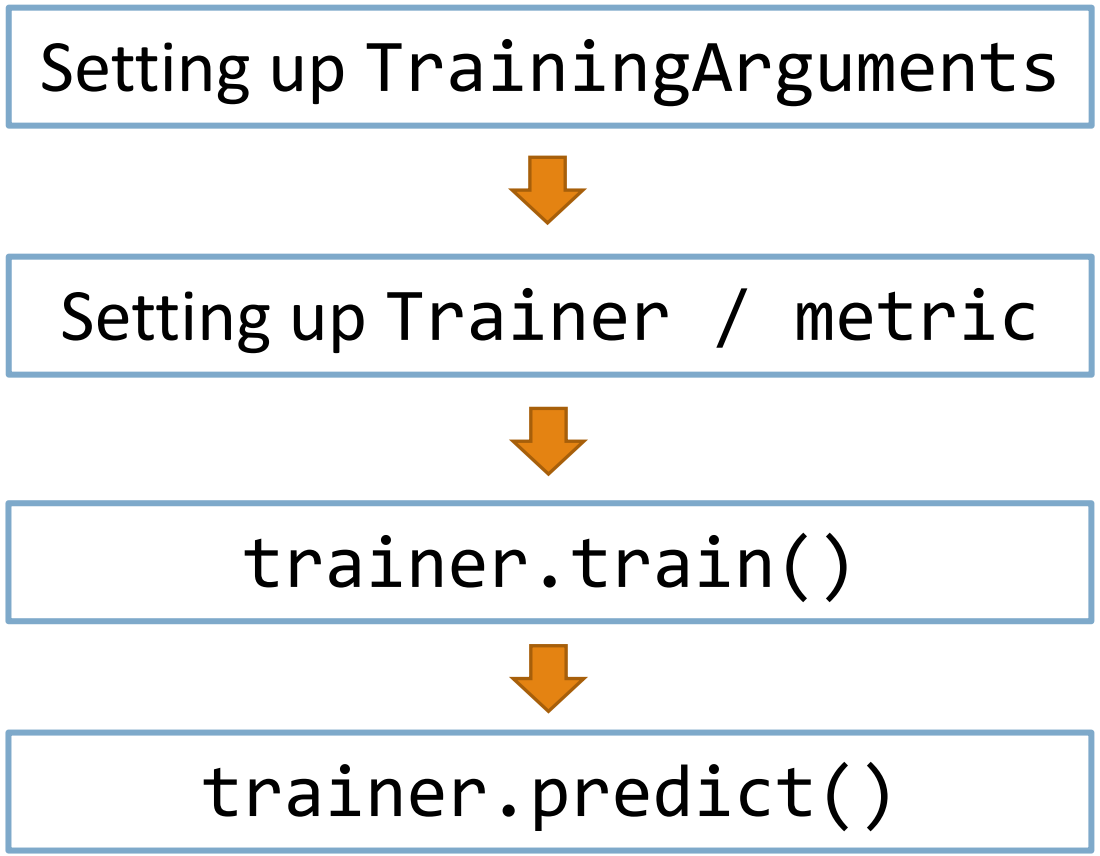
bert output >> label output

What happens for down-stream tasks?

https://github.com/huggingface/transformers/blob/v4.45.2/src/transformers/models/bert/modeling_bert.py#L1715-L1734

```
1651    class BertForSequenceClassification(BertPreTrainedModel):
1675        def forward(
...
1715                if self.config.problem_type is None:
1716                    if self.num_labels == 1:
1717                        self.config.problem_type = "regression"
1718                    elif self.num_labels > 1 and (labels.dtype == torch.long or labels.dtype == torch.int):
1719                        self.config.problem_type = "single_label_classification"
1720                    else:
1721                        self.config.problem_type = "multi_label_classification"
1722
1723                if self.config.problem_type == "regression":
1724                    loss_fct = MSELoss()
1725                    if self.num_labels == 1:
1726                        loss = loss_fct(logits.squeeze(), labels.squeeze())
1727                    else:
1728                        loss = loss_fct(logits, labels)
1729                elif self.config.problem_type == "single_label_classification":
1730                    loss_fct = CrossEntropyLoss()
1731                    loss = loss_fct(logits.view(-1, self.num_labels), labels.view(-1))
1732                elif self.config.problem_type == "multi_label_classification":
1733                    loss_fct = BCEWithLogitsLoss()
1734                    loss = loss_fct(logits, labels)
```

Model Training with Hugging Face Trainer



自動化評估

Trainer can perform training, automatic evaluations, and so on.

Trainer can also help you perform predictions with a trained model.

Why Trainer?

This is a training script with native PyTorch (ref).

```
72      model.train()
73      for epoch in tqdm(range(args.num_epoch)):
74          for batch in tqdm(train_loader):
75              optimizer.zero_grad()
76              input_ids = batch['input_ids'].to(device)
77              attention_mask = batch['attention_mask'].to(device)
78              labels = batch['labels'].to(device)
79              outputs = model(input_ids, attention_mask=attention_mask, labels=labels)
80              loss = outputs[0]
81              loss.backward()
82              optimizer.step()
83
84      model.eval()
85      with torch.no_grad():
86          y_preds = []
87          for batch in tqdm(test_loader):
88              input_ids = batch['input_ids'].to(device)
89              attention_mask = batch['attention_mask'].to(device)
90              labels = batch['labels'].to(device)
91              outputs = model(input_ids, attention_mask=attention_mask, labels=labels)
92              _, predicted = torch.max(outputs[1], 1)
93              y_preds.extend(predicted.tolist())
```

→ `trainer.train()`

Setting up TrainingArguments

```
training_args = transformers.TrainingArguments(
    output_dir='./results',              # output folder
    num_train_epochs=3,                  # number of epochs for training
    learning_rate=2e-5,                  # learning rate
    per_device_train_batch_size=16,      # batch size used for training
    per_device_eval_batch_size=64,       # batch size used for test time
    gradient_accumulation_steps=2,       # training models on bigger batch size
    warmup_steps=500,                    # hyperparameter for learning rate scheduler
    weight_decay=0.01,                   # hyperparameter for optimizer
    evaluation_strategy='steps',         # time unit to perform evaluation
    save_strategy='steps',               # time unit to save checkpoints
    save_steps=500,                      # how often to save checkpoints
    eval_steps=500,                      # how often to perform evaluation
    load_best_model_at_end=True,         # if loading the best checkpoint at the end of training
    metric_for_best_model='eval_loss',   # how to judge the best model
    report_to='tensorboard',             # if saving TensorBoard records
    save_total_limit=10,                 # maximum number of saved checkpoints
    logging_dir='./logs',                # folder for logs
    logging_steps=10,                    # how often to save logs
    seed=random_seed                     # for reproducibility control
)
```

https://huggingface.co/docs/transformers/main_classes/trainer#transformers.TrainingArguments

Build a function for evaluations

```python
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

def compute_metrics(pred):
    labels = pred.label_ids
    preds = np.argmax(pred.predictions, axis=1)
    precision, recall, f1, _ = precision_recall_fscore_support(labels, preds, average='binary')
    acc = accuracy_score(labels, preds)
    return {
        'accuracy': acc,
        'f1': f1,
        'precision': precision,
        'recall': recall
    }
```

eval_loss替換metric

Setting up Trainer

```
trainer = transformers.Trainer(
    model=model, # 🤗 model
    args=training_args, # the `TrainingArguments` you set
    train_dataset=train_dataset, # the training dataset
    eval_dataset=val_dataset, # the evaluation dataset
    compute_metrics=compute_metrics # evaluation metric
)
# Use 1 GPU for training
trainer.args._n_gpu=1

# start training
trainer.train()
```

https://huggingface.co/docs/transformers/main_classes/trainer#transformers.Trainer

Perform predictions after training

```
trainer.predict(test_dataset)
```

- This line will call `compute_metrics` using the `test_dataset`.
- Because we are now using `predict()` with `trainer`, it will not perform model updates.